3D Container Loading: A Step-by-Step Technical Report

Classical heuristic versus a QUBO hybrid, written in plain language

This is a proof-of-concept project. It is a research prototype and not a certified cargo-loading system. The "quantum" part uses a classical simulated annealer, so no quantum hardware was used and no quantum advantage is claimed.

---

1. Summary

What I wanted. Load rectangular packages into one container so that the total weight loaded is as high as possible. Boxes must not overlap, must stay inside the container, and the total weight must stay under a limit. Some boxes may be turned by 90 degrees.

What I did. I did not start with a perfect method. I wrote a simple one, checked it, found what was wrong, and let each problem lead to the next idea. The ideas were:

1. **v1**, a moving pointer that places boxes one by one.
2. **v1.5**, a list of corner points with a scored choice of rotation.
3. **v2**, which keeps track of the **empty space** instead of the boxes.
4. **QUBO v2**, where an annealer chooses *which* packages to load and v2 decides *where* they go.
5. **QUBO v2 with density**, where the energy also rewards packages that carry a lot of weight in little space.

What I found.

- v1 looked great (99.9% of the volume used) but its layouts were **not valid**: boxes overlapped. A checker exposed this.
- v2 gave valid layouts, the same answer every run, and matched the true best answer on every small case I could check with an exact solver.
- The QUBO hybrid also gave valid layouts and stayed small (98 yes/no variables for 80 packages), but it did **not** beat v2 on quality, and it was slower on small and medium test sets.
- Adding density to the energy gave **no clear gain**.

Answer to the main question. *Does the hybrid improve quality, runtime, or neither?* In these tests, **neither**. This is an honest result for a first experiment, and it tells me where to look next.

---

2. The problem in simple words

Imagine a shipping container and a pile of boxes of different sizes and weights. The container can only hold so much volume and so much weight. I cannot load everything, so I must **choose which boxes to load, where to put each one, and which way to turn it**.

Rules (every layout must follow them):

- Every box stays fully inside the container.
- No two boxes share the same space.
- The total weight stays under the weight limit.
- A box may only be turned if it is allowed to turn.

Goal: the highest total weight loaded.

This is a known kind of problem, often called a *container loading problem* or a *3D knapsack*. It is hard because the number of possible arrangements grows extremely fast.

Two ways to measure success

Measure	How it is calculated	Why it matters
Weight %	packed weight divided by the weight limit	This is what my methods try to make large
Volume %	packed volume divided by the container volume	This is what the project brief measures

My methods aim at weight, but the brief counts volume. So I **report both** everywhere. This is an important limit of the study (see Section 8).

---

3. Data and how I tested

3.1 Test data

My first notebook made new random packages every time, so results changed on every run. I fixed this by adding a **seed** (a starting number for the random generator). With the same seed, the same packages are made every time. That way, every method is tested on exactly the same data, and I can repeat any result.

Box sizes are between 10 and 40 units and weights between 1 and 15, as in my original notebook. I made three test sets:

Set	Packages	Container	Package volume as % of container	Total weight	Weight limit	Can rotate
small	12	55 x 55 x 55	89.2%	108	81	9 of 12
medium	20	65 x 65 x 65	107.2%	222	166	14 of 20
large	80	100 x 100 x 100	121.4%	684	800	all 80

- The **large** set is my original setup, and the whole story in Section 4 runs on it.
- On **large**, the total weight (684) is below the limit (800). So on this set, only **space** stops me from loading everything, never the weight limit.
- On **small** and **medium**, the weight limit is 75% of the total weight, so choosing *which* packages to load really matters.

3.2 The checker

When I first looked at v1, a high number (99.9%) looked like a great result. But a 3D picture showed boxes passing through each other. So I built a **checker**: a separate judge that looks at a finished layout, no matter which method made it, and counts:

- boxes that stick out of the container,
- pairs of boxes that overlap,
- total weight over the limit,
- a rotation that is not allowed,
- the same package placed twice.

From then on, no result counted until the checker agreed. The checker is the authority. A method's own claim is only a claim.

3.3 Repeatability

Some methods have no randomness (v1, v1.5, v2). Same input always gives the same output. I confirmed this by running v2 twice and comparing the layouts.

The QUBO method uses an annealer, which is random. So I ran it with **5 seeds (0 to 4)** and recorded every seed. Running the same seed twice gave the same layout. Reporting the mean and the best of 5 runs is fairer than showing one lucky run.

---

4. How the ideas evolved

Each step below follows the same pattern: the thought, the result, what I observed, and the next thought.

Step 1: v1, the moving pointer

Thought. Keep it simple. Sort the packages from heaviest to lightest. Keep one pointer (x, y, z). Place each package at the pointer, then move the pointer along. If a package does not fit, shift the pointer once. If it still does not fit, skip the package.

Result. 64 of 80 packages packed and **99.9% of the volume used**. Weight used: 78.0%.

What I observed. That number is too good for random boxes. In the 3D picture the boxes pass through each other. I saw three problems in my own idea:

1. **Rotation.** It takes the *first* rotation that fits, not the *best* one.
2. **Overlap.** It remembers the pointer but not where the earlier boxes are, so a new box can land inside an old one.
3. **Skipped packages.** A package that fails one shift is dropped for good, even if it would fit elsewhere.

The checker confirmed it:

Test set	Packed	Volume claimed	Overlapping pairs
small	7	62.1%	8
medium	13	67.6%	18
large	64	99.9%	185

Conclusion. v1 gives invalid layouts on all three sets. A high utilisation number means nothing without a checker. The root problem is that v1 forgets the boxes it has already placed.

Next thought. Remember the placed boxes, check for collisions, and choose the rotation by a score.

Step 2: v1.5, corner points and a scored rotation

Thought. Instead of one pointer, keep a **list of corner points** where a box could go. After placing a box, its right, back and top corners become new candidates (these are called *extreme points*). For each package, try every corner with every allowed rotation, throw away options that overlap or leave the container, and pick the best by a fixed score. Ties are broken in a fixed order, so the answer is always the same.

I also added a **support rule**: at least 70% of a box's bottom must rest on the floor or on other boxes, so nothing floats in the air.

Result. 51 of 80 packed, **70.1% volume**, 68.1% weight, **0 violations**.

What I observed. The overlap problem was solved. But only 51 packages fit, and about 30% of the container was still empty, even though the weight limit was not the problem. Packages were rejected although there was room.

My first guess was that the support rule was too strict, so I tested it:

min_support	Packed	Volume %	Weight %	Seconds
0.7	51	70.1	68.1	3.0
0.5	61	68.3	69.2	21.4
0.0 (off)	56	73.2	70.4	6.7

Conclusion. Turning support off helps only a little, and the numbers do not even move in a straight line. So support is **not** the main problem. The real cause is the corner-point list. After each box, only three new points are added. Empty pockets beside short boxes get **no point**, so they are lost forever.

Next thought. Do not remember points. Remember the **empty space itself**.

Step 3: v2, track the empty space

Thought. Keep a list of **free spaces**: empty box-shaped regions. At the start there is one free space, the whole container.

When I place a box in the corner of a free space, I cut it out of every free space it touches. Each cut leaves up to six smaller free spaces (left, right, front, back, below, above). If one free space sits completely inside another, I delete it.

Free spaces are **allowed to overlap each other**. This is on purpose: it means no empty pocket is ever lost.

A small example in 2D. The container is 10 wide and 10 tall. I place a 4 by 3 box in the bottom-left corner. Two free spaces remain: the strip *above* the box (10 wide, 7 tall) and the strip to the *right* of it (6 wide, 10 tall). They overlap in the top-right corner, and that is fine. A later box can use whichever one suits it. In 3D the same thing happens with up to six pieces.

How a box is placed. For every free space and every rotation that fits inside it, I compute a **best-fit score**: how much room would be wasted. The lowest score wins. So **position and rotation are decided together**, which answers my first doubt about v1.

Giving skipped packages more chances.

- v2 runs **12 passes**: 4 orderings of the packages (by density, by weight, by volume, by longest side) times 3 scoring rules. It keeps the pass that loads the most weight.
- Then a **promotion** step takes a package that was skipped, moves it to the front of the order, and runs again. The change is kept only if total weight improves.

Nothing in v2 is random.

Result.

Version	Packed	Volume %	Weight %	Violations
v1	64	99.9 (not valid)	78.0	185
v1.5	51	70.1	68.1	0
v2	**63**	**87.2**	**76.6**	**0**

Running v2 twice gives the identical layout, so it really is deterministic. All three doubts about v1 are handled: rotation by score, overlap impossible by construction (and the checker agrees), and skipped packages get more chances.

Step 3a: What "support" costs in v2

Support means: for a box above the floor, how much of its bottom rests on the floor or on other boxes. 1.0 means fully resting. 0.5 means half of it hangs in the air. `min_support` is the lowest share I allow.

min_support	Packed	Volume %	Weight %
0.0 (off)	63	87.2	76.6
0.5	62	84.7	76.0
0.7	61	81.6	74.0

Stricter stability costs about 2.5 points of volume at 0.5 and about 5.6 points at 0.7. The brief says load-bearing physics is out of scope, so I keep support **off** in the experiments. It is a switch for real use. Note that it only checks the *area* under a box, not whether the box below is strong enough.

Step 3b: What if I only use the density order?

Thought. Density is weight divided by volume. It says how much weight a package carries for the space it takes. If my goal is weight, the densest packages should be the best ones to load. And one ordering would be simpler than four.

Result.

Version	Packed	Volume %	Weight %
v2 (4 orderings x 3 scoring rules)	63	87.2	76.6
v2, density order only	65	79.2	74.1

Conclusion. Density-only packs *more* packages but *less* weight and *less* volume. The density order loads many small packages first. They fill the room but do not carry the most weight. More packages does not mean more weight. I **dropped** this idea. The lesson: an idea that sounds right still has to be tested.

Step 3c: How good is v2, really?

Thought. "87.2%" means little if I do not know the best possible answer. So I need a yardstick. I used an **exact solver** (OR-Tools CP-SAT). On small cases laid out on a 10-unit grid, it finds the **true maximum weight**.

Result. (These five small cases use a 40 x 40 x 30 container.)

Packages	Weight limit	Exact best	v1.5	v2
8	45	45	43	**45**
10	60	60	59	**60**
12	70	70	69	**70**
14	80	80	80	**80**
16	90	90	90	**90**

Conclusion. v2 matched the true best on all 5 cases, while v1.5 missed three of them by 1 or 2 units. On small cases v2 is already optimal. On the big dataset I cannot prove it, but it is strong. Only 5 small cases were tested, so this is a good sign, not a proof. **v2 became my finalised classical method.**

Next thought. v2 is already good at geometry (where a box goes and how it is turned). But one part of the problem is a pure **selection** question: *which packages should I load* so the total weight is highest while the limits hold? That is a knapsack problem, and knapsack problems are what a QUBO and an annealer are made for. So the plan: let the annealer choose **which** packages, and let v2 decide **where and how turned**.

5. The QUBO hybrid (the "quantum-inspired" part)

5.1 What is a QUBO?

A QUBO describes a problem with **yes/no variables** and **one energy formula**. The lowest energy is the best answer. A QUBO has **no hard rules**, so every rule has to become a **penalty**: a big energy cost for breaking it.

The yes/no variables are called **qubits** here. In this project a qubit is simply one yes/no choice.

5.2 My QUBO: one yes/no choice per package

For each package *i*, one variable: `x_i = 1` means "load this package", `x_i = 0` means "leave it".

Energy term	What it does
minus the reward of chosen packages	rewards heavy loads (the reward is the weight, at first)
weight-limit penalty	total weight must stay under the limit
volume-budget penalty	total volume must stay under a budget
a few extra "slack" qubits	let the two limits be written as simple sums

Slack qubits, in simple words. A rule like "total weight is at most 800" is not easy to write as a plain sum. So I add a few extra yes/no variables that soak up the leftover room. Then the rule becomes "weight plus leftover equals 800", which can be written as a penalty.

The volume budget. The budget is a fraction of the container volume (I try 1.0, 0.9, 0.8 and 0.7). It is only a **rough guess** of "will these packages fit?". The true answer depends on geometry, and that is decided later by v2.

Penalty size. Every penalty is set larger than the best single reward, so breaking a limit never pays.

5.3 The annealer

An **annealer** searches for the lowest-energy answer. It works a bit like slowly cooling metal: early on it accepts many random changes, later it accepts only good ones. Here it is **simulated annealing running on a normal computer** (the `neal` library). It sits behind one function call, so a different solver could replace it later.

5.4 The hybrid pipeline

1. The annealer picks a set of packages. This is only a **candidate**.
2. A classical **repair** step fixes the pick if it breaks a limit (it removes packages that give little weight for the space they use).
3. The chosen packages go **first** (biggest first) into v2's free-space placement. The remaining packages follow by density as a fallback. So the layout is always geometrically valid.

4. The **checker** has the last word.

This is why it is called a *hybrid*: a QUBO and an annealer do the choosing, and classical code does the placing, the repair and the checking.

5.5 Results: QUBO v2 with weight only

On the large dataset with 5 seeds:

Seed	Packed	Volume %	Weight %	Total seconds	Annealer seconds	Qubits
0	66	77.6	72.8	5.9	3.9	98
1	65	82.1	73.2	5.3	3.6	98
2	66	78.8	73.5	5.4	3.5	98
3	65	79.5	73.2	5.6	3.7	98
4	68	87.2	72.6	5.4	3.5	98

On the five small cases (seed 0) it got 45, 58, 69, 80, 90 against the exact best of 45, 60, 70, 80, 90.

What I observed.

- It is **small**: only 98 qubits for 80 packages. Every layout was valid.
- It packs **more packages** (65 to 68, against 63 for v2), but the **weight is lower** (72.6 to 73.5%, against 76.6% for v2).
- The volume **changes a lot between seeds** (77.6% to 87.2%). v2 never changes.
- The annealer takes most of the time.

Conclusion. The idea works and stays small, but the quality is not better than v2. The annealer only sees weight and a rough volume budget, so it cannot know which selection will really pack well. And v2 already makes the selection well through its orderings and its promotion step.

Next thought. Weight alone says nothing about how much room a package takes. A package that gives a lot of weight in little space should be preferred. So I put **density** into the energy.

5.6 Results: QUBO v2 with weight and density

The reward changes from `weight` to:

`reward = weight x (1 + lambda x density / highest density)`, where `density = weight / volume`.

With `lambda = 0` it is the old version. A bigger `lambda` pushes the annealer to prefer dense packages. I tested `lambda` = 0, 0.5, 1, 2, 4 on **10 small cases**, and on the large dataset.

Total weight on the 10 small cases (higher is better, the exact best is the ceiling):

Exact best	v2	lambda 0	lambda 0.5	lambda 1	lambda 2	lambda 4
650	650	647	643	644	646	646

Large dataset, seed 0:

lambda	Packed	Volume %	Weight %
0	66	77.6	72.8
1	64	76.2	72.2
2	67	82.8	74.4

Conclusion. Density did **not** help clearly. On the small cases, the differences (1 to 4 units) are about the size of the random seed noise. On the large set there is no clear trend. A likely reason is that the volume budget already pushes the annealer towards packages that use little room, so density says the same thing twice, and v2 already has its own density ordering. I kept `lambda` as a switch, but I have **no evidence it helps**.

---

6. The final fair comparison

6.1 Setup

Four methods, the same three test sets:

- **A**: v1
- **B**: v2 (finalised classical method)
- **C**: QUBO v2, weight only
- **D**: QUBO v2, weight plus density (`lambda = 1`)

A and B are deterministic, so they ran once. C and D ran with 5 seeds each. Every layout went through the checker.

6.2 Results (mean over runs)

Test set	Method	Packed	Volume % (mean / best)	Weight % (mean / best)	Violations
small	A v1	7	62.1	100.0	**8**
	B v2	8	62.7	100.0	0
	C QUBO weight	8.2	62.2 / 66.3	99.3 / 100.0	0
	D QUBO weight+density	8.0	58.3 / 61.3	99.5 / 100.0	0
medium	A v1	13	67.6	99.4	**18**
	B v2	13	71.1	100.0	0
	C QUBO weight	14.2	70.2 / 78.2	99.9 / 100.0	0

Test set	Method	Packed	Volume % (mean / best)	Weight % (mean / best)	Violations
	D QUBO weight+density	14.0	69.9 / 73.7	99.6 / 100.0	0
large	A v1	64	99.9	78.0	**185**
	B v2	63	87.2	76.6	0
	C QUBO weight	66.0	81.0 / 87.2	73.1 / 73.5	0
	D QUBO weight+density	65.2	81.1 / 86.0	73.6 / 75.1	0

On the large set, the annealer itself took about 3.9 s (C) and 3.7 s (D) of those totals.

6.3 The whole journey in one table (large dataset)

Version	Packed	Volume %	Weight %	Violations
v1	64	99.9	78.0	185
v1.5 extreme points	51	70.1	68.1	0
v2	63	87.2	76.6	0
v2 density order only (dropped)	65	79.2	74.1	0
QUBO v2, weight only (seed 0)	66	77.6	72.8	0
QUBO v2 with density (seed 0)	64	76.2	72.2	0

---

7. What I learned

Step	What I tried	What it taught me
1	v1	A great-looking number can be false. Build the checker first.
2	Corner points	Fixing overlap is not enough. How I remember free space matters.
3	v2	Remember the **empty space**. This gave valid, repeatable, 87% results.
3b	Density-only order	A sensible idea can still lose. Test it.

Step	What I tried	What it taught me
3c	Exact solver	v2 is already optimal on small cases, so the hybrid has a hard target to beat.
4	QUBO v2, selection	Small and valid, but not better than v2.
5	Density in the energy	No clear gain.

Answer to the brief's question. The hybrid improved **neither quality nor runtime** over the classical v2. v2 matched or beat it on weight, was faster on small and medium sets, and gave the same answer every time. The hybrid's one visible advantage is that different seeds give different valid layouts, and on the medium set some seeds reached higher volume (up to 78.2% against 71.1%). But other seeds were worse, so I cannot call that an improvement. Nothing here shows a quantum advantage, and I did not expect one.

NOTE: But I cant say for sure coz I looked it in one perspective took weight into consideration and the formulated certain penaltities and ran it in a classical annealer simulator , so if time permits I can look into other ways of the quantum classical hybrid approach if there is ONE…
